

Tập luận văn đội tuyển quốc gia Trung Quốc năm 2013

Huấn luyện viên: Hu Weidong, Tang Wenbin

Biên tập: Hu Weidong

China Computer Federation, 2013

Nguồn gốc: Tập luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2013

IOI China candidate team papers / National training team 2013 collection.pdf

Tóm tắt

PDF nguồn là tập hợp luận văn đội tuyển quốc gia Trung Quốc năm 2013. Bản dịch này chuyển ngữ thông tin đầu sách, mục lục và bốn bài đầu tiên: *Kế hoạch lên đỉnh*, của Peng Tianyi, *Lấy số trên lưới*, của Wang Kangning, *Thảo luận bài Two strings*, của Luo Gan, và *Catch The Penguins*, của Zhang Wentao. Trạng thái hiện tại: bốn bài đầu đã được dịch; các bài còn lại mới có mục lục. Lớp văn bản của phần thân bị lỗi ánh xạ phong chữ, nên phần bài viết được đối chiếu từ OCR trang render và các công thức, ví dụ còn đọc được trong lớp văn bản.

1 Thông tin đầu sách

Tập sách có tiêu đề *Tập luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2013*. Huấn luyện viên là Hu Weidong và Tang Wenbin; biên tập là Hu Weidong. Đơn vị xuất bản ghi trên bìa là China Computer Federation.

2 Mục lục đã dịch

Trang	Bài viết	Tác giả / trường
1	Kế hoạch lên đỉnh	Peng Tianyi, Trường THPT trực thuộc Đại học Sư phạm Hồ Nam
9	Lấy số trên lưới	Wang Kangning, Trường THPT trực thuộc Đại học Sư phạm Đông Bắc, Cát Lâm
17	Thảo luận bài <i>Two strings</i>	Luo Gan, Trường Học Quân Hàng Châu, Chiết Giang
23	<i>Catch The Penguins</i>	Zhang Wentao, Trường Học Quân Hàng Châu, Chiết Giang
27	Bàn về ứng dụng tư tưởng chia khối trong một lớp bài xử lý dữ liệu	Luo Jianqiao, Trường THPT số 80 Bắc Kinh
43	Kỹ thuật <i>meet in the middle</i> trong bài toán tìm kiếm	Qiao Mingda, Trường Ngoại ngữ Nam Kinh, Giang Tô
57	Phân tích cơ sở và ứng dụng của xác suất trong thi tin học	Hu Yuanming, Trường Trung học Dương Châu, Giang Tô
71	Bàn về một số cách giải không kinh điển cho bài cấu trúc dữ liệu	Xu Haoran, Trường Ngoại ngữ Nam Kinh, Giang Tô
85	Ứng dụng cây cân bằng theo trọng lượng và cây cân bằng hậu tố trong Olympic Tin học	Chen Lijie, Trường Ngoại ngữ Hàng Châu, Chiết Giang
99	Bàn về bài toán đếm trên vòng	Gao Shenghan, Trường Trung học cao cấp Thường Châu, Giang Tô
109	Ứng dụng của phương pháp chia khối	Wang Ziyu, Trường Trung học số 1 Shengli, Sơn Đông
121	Bàn về nguyên lý bao hàm–loại trừ	Wang Di, Trường Trung học số 7 Thành Đô, Tứ Xuyên

3 Trạng thái và ghi chú trích xuất

Các trang in trong mục lục lệch 4 trang so với trang vật lý của PDF: trang vật lý 5 là trang in 1. Bài *Kế hoạch lên đỉnh* dùng các trang in 1–8, tức trang vật lý 5–12. Bài *Lấy số trên lưới* dùng các trang in 9–16, tức trang vật lý 13–20. Bài *Thảo luận bài Two strings* bắt đầu ở trang in 17, tức trang vật lý 21; bài kế tiếp bắt đầu ở trang in 23, tức trang vật lý 27. Trang in 22 là trang trắng, nên phần dịch bài thứ ba lấy nội dung ở các trang in 17–21. Bài *Catch The Penguins* dùng các trang in 23–26, tức trang vật lý 27–30.

4 Kế hoạch lên đỉnh

Peng Tianyi
Trường THPT trực thuộc Đại học Sư phạm Hồ Nam

4.1 Bối cảnh bài toán

Leo núi là một môn vận động rất có ích cho sức khỏe thể chất và tinh thần. Nhưng khi leo núi, ta nên chọn chiến lược như thế nào để lên tới đỉnh núi nhanh nhất? Từ câu hỏi thú vị này, tác giả nghĩ ra bài toán dưới đây.

4.2 Mô tả bài toán

Giới hạn tài nguyên. Giới hạn thời gian: 2 giây. Giới hạn bộ nhớ: 512 MB.

Phát biểu. Trên mặt phẳng hai chiều, một dãy núi được xác định bởi một dãy đỉnh

$$(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n).$$

Hai đỉnh kề nhau được nối bằng đoạn thẳng, với bảo đảm x_i tăng nghiêm ngặt. Như vậy ta thu được một dãy núi liên tục; giá trị y của mỗi điểm là độ cao của điểm đó.

Ta định nghĩa phần trong của dãy núi là phần kẹp giữa đường gấp khúc nối các đỉnh và trục x , không tính chính đường gấp khúc đó. Nếu đoạn thẳng nối đỉnh A và đỉnh B không đi qua phần trong của dãy núi, ta nói đỉnh A nhìn thấy đỉnh B , hoặc đỉnh B nhìn thấy đỉnh A .

Bây giờ pty xuất phát từ một đỉnh nào đó và muốn lên tới đỉnh cao nhất của dãy núi. Anh chỉ có thể đi trên đường gấp khúc giữa các đỉnh. Sau khi suy nghĩ, anh chọn cách leo như sau:

1. Đứng tại điểm xuất phát và nhìn sang hai bên. Nếu đỉnh núi cao nhất nhìn thấy được lúc này nằm bên trái, anh đi sang trái; ngược lại anh đi sang phải.
2. Trong lúc đi, pty vẫn quan sát đỉnh cao nhất nhìn thấy được ở hai bên. Một khi phát hiện một đỉnh cao hơn tất cả những đỉnh đã thấy trước đó, anh sẽ đi về phía đỉnh cao nhất lúc này.
3. Nếu có một thời điểm mà vị trí pty đang đứng cao hơn mọi đỉnh cao nhất nhìn thấy được ở hai bên, thì anh đã tới đỉnh cao nhất của dãy núi và quá trình leo núi kết thúc.

pty muốn biết: nếu dùng chiến lược trên và xuất phát từ từng đỉnh, quãng đường để tới đỉnh cao nhất lần lượt là bao nhiêu? Khoảng cách giữa hai điểm trên mặt phẳng là độ dài đoạn thẳng nối chúng.

Dữ liệu vào. Dòng đầu tiên chứa một số nguyên n , là số đỉnh của dãy núi.

Tiếp theo là n dòng; dòng thứ i chứa hai số nguyên x_i, y_i , biểu diễn tọa độ của đỉnh thứ i . Bảo đảm x_i tăng nghiêm ngặt, các y_i đôi một khác nhau trừ y_1, y_n , và x_i, y_i đều là số nguyên không âm. Bảo đảm $y_1 = y_n = 0$.

Dữ liệu ra. In ra n dòng, mỗi dòng một số thực.

Số thực ở dòng thứ i biểu diễn quãng đường từ đỉnh thứ i tới đỉnh cao nhất. Nếu sai số giữa kết quả in ra và đáp án chuẩn không vượt quá 10^{-2} , test đó được điểm tối đa; ngược lại được 0 điểm.

Ví dụ vào.

```
6
0 0
1 6
2 4
3 1
```

5 5
6 0

Ví dụ ra.

6.08
0.00
2.24
6.52
9.87
14.97

Giải thích ví dụ. Các lộ trình là:

$A : A \rightarrow B,$
 $B : B,$
 $C : C \rightarrow B,$
 $D : D \rightarrow G \rightarrow D \rightarrow C \rightarrow B,$
 $E : E \rightarrow D \rightarrow C \rightarrow B,$
 $F : F \rightarrow E \rightarrow D \rightarrow C \rightarrow B.$

Khi xuất phát từ D , đỉnh cao nhất nhìn thấy được ban đầu là E . Khi đi tới điểm G , ptý phát hiện đỉnh cao hơn là B , nên quay đầu và tiếp tục đi về phía B . Với các điểm xuất phát còn lại, lộ trình không cần đổi hướng.

Quy mô dữ liệu và quy ước. Mọi dữ liệu đều thỏa mãn $x_i, y_i \leq 100000$.

Test	Ràng buộc
1–4	$n \leq 20$
5–8	$n \leq 70$
9–10	$n \leq 100000$, và mọi đỉnh đều nhìn thấy trực tiếp đỉnh cao nhất
11–14	$n \leq 30000$
15–20	$n \leq 100000$

4.3 Phân tích bài toán

4.3.1 Đơn giản hóa bài toán

Bài toán yêu cầu tại mọi thời điểm khi leo núi ta đều phải quan sát đỉnh cao nhất hiện tại ở hai bên. Trước hết hãy tạm đơn giản hóa vấn đề: chỉ khi tới một đỉnh ta mới quan sát đỉnh cao nhất lúc đó. Nói cách khác, điểm quan sát tạm thời chuyển từ vô hạn nhiều điểm thành hữu hạn $O(n)$ đỉnh.

4.3.2 Tìm đỉnh cao nhất mà một đỉnh nhìn thấy

Gọi đỉnh cao nhất mà đỉnh i nhìn thấy được về bên trái là $L[i]$, và đỉnh cao nhất mà nó nhìn thấy được về bên phải là $R[i]$. Hãy tưởng tượng ta đứng tại một điểm nào đó; ban đầu tia nhìn nằm ngang về bên trái, sau đó ta từ từ ngẩng đầu lên cho tới khi tia nhìn không còn bị một ngọn núi nào che khuất. Khi đó ta tìm được $L[i]$.

Giả sử đỉnh hiện tại là A , còn $L[i]$ là B . Hai tính chất sau là hiển nhiên:

- i điểm đầu đều nằm phía dưới đoạn AB , theo nghĩa không nghiêm ngặt;
- A và B là hai điểm kề nhau cuối cùng trên bao lồi trên của i điểm đầu.

Vì A chắc chắn là điểm cuối cùng của bao lồi trên của i điểm đầu, B chính là điểm áp chót trên bao lồi trên đó.¹ Do đó ta chỉ cần dùng một stack để duy trì bao lồi của các điểm đầu tiên. Mỗi lần thêm điểm thứ $i + 1$, ta đồng thời duy trì tính chất của bao lồi. Vì tổng cộng chỉ có $O(n)$ đỉnh, ta có thể tìm toàn bộ $L[i]$ và $R[i]$ trong thời gian $O(n)$.

4.3.3 Xây dựng mô hình đồ thị

Cách làm vét cạn. Sau khi có $L[i]$ và $R[i]$, ta có một thuật toán khá thô: với từng đỉnh xuất phát, mô phỏng sự thay đổi của lộ trình. Độ phức tạp là $O(n^2)$ hoặc cao hơn. Điều này cho thấy ta cần tiếp tục khai thác phần thông tin bị tính lặp.

Dựng cây. Gọi $H[i]$ là độ cao của đỉnh thứ i , và đặt

$$T[i] = \max(H[L[i]], H[R[i]]).$$

Khi đó $T[i]$ biểu diễn độ cao của đỉnh cao nhất mà điểm i nhìn thấy được.

Khi ta xuất phát từ A , đến điểm đầu tiên B thỏa mãn

$$T[B] \geq T[A],$$

thì đoạn đường còn lại cần đi sẽ hoàn toàn giống như khi xuất phát từ B . Vì vậy khoảng cách từ A tới đỉnh cao nhất bằng khoảng cách từ B tới đỉnh cao nhất cộng thêm khoảng cách từ A tới B .

Lúc này cho B nối tới A bằng một cạnh có hướng, trọng số là độ dài AB . Làm như vậy với mọi đỉnh, ta thu được một cây: đỉnh cao nhất không có bậc vào, còn mọi đỉnh khác có bậc vào bằng 1. Tổng trọng số trên đường đi từ gốc cây tới một đỉnh bất kỳ chính là quãng đường từ đỉnh đó tới đỉnh cao nhất. Chỉ cần DFS một lần là có đáp án cho mọi đỉnh.

Vấn đề cần giải quyết. Với một điểm A , làm thế nào nhanh chóng tìm được điểm đầu tiên về bên trái hoặc bên phải thỏa mãn $T[B] \geq T[A]$? Đây là một bài toán cấu trúc dữ liệu rất kinh điển. Ta có thể giải bằng nhị phân đáp án kết hợp RMQ trên đoạn. Dưới đây là một cách có hằng số nhỏ hơn.

Lập danh sách liên kết hai chiều của toàn bộ đỉnh theo thứ tự từ trái sang phải. Sau đó duyệt tất cả các điểm theo thứ tự T tăng dần. Mỗi khi duyệt tới một điểm A , điểm đầu tiên bên trái thỏa mãn $T[B] \geq T[A]$ chính là hàng xóm bên trái của A còn tồn tại trong danh sách liên kết tại thời điểm đó. Nối cạnh tương ứng, rồi xóa A khỏi danh sách liên kết. Phía bên phải được xử lý tương tự. Độ phức tạp của thuật toán này chính là độ phức tạp sắp xếp.

4.3.4 Quay lại bài toán ban đầu

Tới đây ta đã giải xong phiên bản đơn giản hóa trong thời gian sắp xếp $O(n \log n)$. Bây giờ hãy quay lại bài toán ban đầu.

Ta có khả năng quan sát đỉnh cao nhất tại mọi thời điểm. Điều này làm bài toán thay đổi như thế nào?

Giả sử hiện tại ta xuất phát từ điểm A , đi sang phải, tới một điểm B trên một đoạn nào đó thì phát hiện một đỉnh cao hơn ở bên trái, nên quay sang đi về bên trái. Điểm rẽ B phải có những tính chất gì? Gọi đoạn chứa B là P_1P_2 . Khi đó B thỏa mãn:

¹Một kết luận đẹp như vậy thật sự làm tác giả ngạc nhiên.

- B là giao điểm giữa P_1P_2 và đường thẳng nối hai điểm nào đó bên trái A , ký hiệu là T_1, T_2 ;
- các điểm trước A đều nằm phía dưới đường thẳng T_1T_2 , theo nghĩa không nghiêm ngặt;
- T_1, T_2 là hai điểm kề nhau trên bao lồi trên của các điểm trước A .

Điều này có nghĩa: một điểm quan sát có ý nghĩa B sẽ là giao điểm đầu tiên giữa đường kéo dài của một cạnh nào đó trên bao lồi trên và đường gấp khúc của dãy núi. Vì tổng số cạnh được sinh ra trong quá trình duy trì bao lồi trên là $O(n)$, số điểm quan sát có ý nghĩa cũng là $O(n)$.²

Vậy làm thế nào nhanh chóng tìm ra các điểm quan sát đó?

Nếu đường thẳng T_1T_2 cắt đoạn P_1P_2 , điều này có nghĩa là khi P_2 được thêm vào bao lồi, T_2 sẽ bị bật ra; còn khi P_1 được thêm vào bao lồi thì T_2 chưa bị bật ra. Vì thế trong lúc duy trì bao lồi, ta chỉ cần lấy giao điểm giữa các đường thẳng liên quan tới điểm bị bật ra và điểm vừa chèn, là có thể thu được toàn bộ điểm quan sát. Độ phức tạp vẫn là $O(n)$.

Bài toán với hữu hạn điểm quan sát đã được giải ở trên. Vì thế toàn bộ bài toán được giải trọn vẹn trong thời gian $O(n \log n)$.

4.4 Tổng kết

4.4.1 Nguồn cảm hứng

Cảm hứng của bài toán đến từ suy nghĩ về việc leo núi trong đời sống thực. Dùng kiến thức thuật toán để phân tích một vấn đề thực tế không chỉ nâng cao năng lực phân tích thuật toán, mà còn phản ánh mục đích cốt lõi của việc học là “học để dùng”. Nó cũng có thể khơi dậy hứng thú đối với thi Tin học và làm ta thấy vui trong quá trình đó. Hy vọng cách suy nghĩ này đem lại gợi ý cho mọi người.

4.4.2 Tư duy giải bài

Nhìn lại quá trình giải bài: trước hết ta đơn giản hóa bài toán, biến vô hạn thành hữu hạn; tiếp đó chia bài toán thành các phần nhỏ hơn và lần lượt giải quyết. Đây đều là những phương pháp thường dùng trong quá trình làm bài.

Dưới đây là vài cảm nhận của tác giả về phương pháp giải bài, xin chia sẻ với mọi người.

Trong quá trình giải bài thông thường, ta cần dựng một chiếc cầu giữa điều kiện đã biết và vấn đề cần tìm. Có thể dùng công cụ đã học để suy luận xuôi từ điều kiện đã biết, xem ta còn thu được gì. Cũng có thể suy luận ngược từ vấn đề cần tìm, xem ta còn cần biết gì. Khi cây cầu suy luận xuôi và cây cầu suy luận ngược có một điểm trùng nhau, cây cầu nối từ điều kiện tới bài toán đã được dựng xong, và bài toán cũng được giải quyết.

Tuy nhiên phương pháp giải bài biến hóa muôn hình vạn trạng; không tồn tại bí kíp vạn năng nào. Điều đáng mừng là: khi số bài bạn đã giải càng nhiều, khả năng bạn giải được bài tiếp theo càng lớn. Vì vậy, chỉ cần chăm luyện tập và chịu suy nghĩ, năng lực giải bài chắc chắn sẽ không ngừng bước lên những nấc thang mới.

5 Lấy số trên lưới

Wang Kangning

Trường THPT trực thuộc Đại học Sư phạm Đông Bắc, Cát Lâm

²Đây cũng là một tính chất gây phần khích.

Tóm tắt

Đây là báo cáo lời giải cho một bài toán thú vị trong đợt làm đề đối kháng của đội tuyển quốc gia năm 2013.

5.1 Đề bài

5.1.1 Mô tả bài toán

Cho một bảng ô vuông $N \times N$. Mỗi ô chứa một số; số ở hàng i , cột j bằng $A_i \cdot B_j$.

Một người đi từ góc trên bên trái của một hình chữ nhật con tới góc dưới bên phải của hình chữ nhật đó. Mỗi bước chỉ được đi sang phải hoặc đi xuống một ô. Anh ta cộng các số đi qua trên đường đi và muốn tổng này nhỏ nhất.

Anh ta quyết định đi T lần. Lần thứ i , anh đi từ ô hàng P_i , cột Q_i tới ô hàng R_i , cột S_i . Hãy hỏi tổng nhỏ nhất của các số đi qua trong mỗi lần.

5.1.2 Dữ liệu vào

Dòng đầu tiên gồm hai số nguyên N, T , lần lượt là kích thước bảng và số lượng truy vấn.

Dòng thứ hai gồm N số nguyên $A_1, A_2, \dots, A_N$. Dòng này được sinh ngẫu nhiên.

Dòng thứ ba gồm N số nguyên $B_1, B_2, \dots, B_N$. Dòng này cũng được sinh ngẫu nhiên.

Tiếp theo là T dòng, mỗi dòng gồm bốn số nguyên P_i, Q_i, R_i, S_i , có ý nghĩa như trên và thỏa mãn $P_i \leq R_i, Q_i \leq S_i$.

5.1.3 Dữ liệu ra

In ra T dòng; mỗi dòng là một số nguyên, biểu diễn đáp án của truy vấn tương ứng.

5.1.4 Ví dụ vào

```
3 4
1 2 3
2 3 4
1 1 1 1
1 3 1 3
1 1 3 3
1 1 3 1
```

5.1.5 Ví dụ ra

```
2
4
29
12
```

5.1.6 Phạm vi dữ liệu

- Với 20% số test: $N \leq 100, T \leq 100$.

- Với 50% số test: $N \leq 3000$, $T \leq 1000$.
- Với 70% số test: $N \leq 50000$, $T \leq 20000$.
- Với 100% số test: $N \leq 200000$, $T \leq 50000$, $1 \leq A_i, B_i \leq 10^6$.

Bảo đảm mọi truy vấn đều hợp lệ. Ngoài ví dụ, tất cả các giá trị A_i, B_i đều được sinh ngẫu nhiên.

Giới hạn tài nguyên. Giới hạn thời gian là 2 giây, giới hạn bộ nhớ là 512 MB.

5.2 Hướng giải

5.2.1 Thuật toán một

Với mỗi truy vấn, dùng quy hoạch động trực tiếp trên toàn bộ hình chữ nhật của truy vấn.

Độ phức tạp thời gian là $O(T \cdot N^2)$, độ phức tạp bộ nhớ là $O(N^2)$. Cách này chỉ kỳ vọng đạt khoảng 20 điểm.

5.2.2 Thuật toán hai

Thuật toán trên trả lời truy vấn quá chậm, nên ta cần tăng tốc phần truy vấn.

Xét tư tưởng chia để trị. Mỗi lần ta dùng đường giữa để chia hình chữ nhật thành hai nửa, dùng quy hoạch động tính đáp án từ từng điểm trên đường giữa tới từng điểm trong hình chữ nhật hiện tại, rồi đệ quy xử lý hai hình chữ nhật ở hai bên đường giữa. Nếu điểm đầu và điểm cuối của một truy vấn nằm ở hai phía của đường giữa, ta liệt kê điểm trên đường giữa mà đường đi đi qua; nếu không, truy vấn được đẩy xuống một hình chữ nhật con.

Nếu lần nào cũng cắt theo đường thẳng đứng, đặt độ phức tạp tiền xử lý là $\text{Time}(x)$, ta có

$$\text{Time}(x) = 2 \text{Time}(x/2) + O(N^2x).$$

Khi đó độ phức tạp tiền xử lý là $O(N^3 \log N)$. Nếu luân phiên cắt theo chiều ngang và chiều dọc, ta có

$$\text{Time}(x) = 2 \text{Time}(x/2) + O(x^3),$$

nên độ phức tạp tiền xử lý là $O(N^3)$.

Tổng độ phức tạp thời gian là $O(N^3 + TN)$, độ phức tạp bộ nhớ là $O(N^3)$. Chi phí tiền xử lý vẫn quá cao, vì vậy cần tìm phương án khác.

5.2.3 Thuật toán ba

Xét thuật toán chia khối. Nếu chia hình vuông thành các khối $S \times S$, ta tiền xử lý đáp án từ mỗi điểm tới mỗi điểm trên biên khối, đồng thời tiền xử lý đáp án giữa mọi cặp điểm trong cùng một khối.

Khi trả lời truy vấn, nếu hai điểm không nằm trong cùng một khối, ta liệt kê vị trí đầu tiên mà đường đi đi qua biên khối; nếu chúng nằm trong cùng một khối thì trả lời trực tiếp.

Độ phức tạp thời gian là

$$O\left(N^2 \left(S + \frac{N}{S^2}\right) + T \cdot \frac{N}{S}\right),$$

và độ phức tạp bộ nhớ là

$$O\left(N^2 \left(S + \frac{N}{S^2}\right)\right).$$

Độ phức tạp này cũng còn quá cao.

5.2.4 Cải tiến

Một nguyên nhân chính khiến các thuật toán trên kém hiệu quả là chúng chưa dùng tới điều kiện quan trọng trong đề: hai dãy A và B đều được sinh ngẫu nhiên.

Qua thử nghiệm và quan sát, ta thấy số điểm rẽ của đường đi tối ưu không nhiều. Hãy xét một điểm rẽ và hai đoạn đường đi kề với nó. Nếu điểm rẽ nằm ở một cột có trọng số không nhỏ hơn một cột nào đó bên trái trong vùng đang xét, đồng thời cũng không nhỏ hơn một cột nào đó bên phải, thì luôn tồn tại một cách biến đổi đường đi không tệ hơn và làm mất điểm rẽ đó.

Cụ thể, giả sử trong hình minh họa của bài gốc, cột thứ năm trong khung có trọng số không nhỏ hơn trọng số của cột ngoài cùng bên trái và cột ngoài cùng bên phải của khung. Ta xét hai cách biến đổi đường đi quanh điểm rẽ:

- Nếu trọng số của hàng phía trên trong vùng nhỏ hơn trọng số của hàng phía dưới, dùng biến đổi thứ nhất sẽ cho đáp án tốt hơn.
- Nếu trọng số của hàng phía trên lớn hơn trọng số của hàng phía dưới, dùng biến đổi thứ hai sẽ cho đáp án tốt hơn.
- Nếu hai trọng số hàng bằng nhau, dùng cách biến đổi nào thì đáp án cũng không xấu đi.

Như vậy ta có thể loại bỏ điểm rẽ đang xét.

Từ lập luận trên suy ra: với một điểm rẽ bất kỳ, trọng số của hàng hoặc cột chứa nó phải nhỏ hơn tất cả trọng số của các hàng hoặc cột ở một phía trong vùng truy vấn. Nói cách khác, nếu nhìn theo một phía của đoạn, điểm rẽ phải là một kỷ lục nhỏ mới.

Với dữ liệu ngẫu nhiên, xác suất để giá trị thứ i nhỏ hơn mọi giá trị đứng trước nó là $1/i$. Vì vậy số lượng điểm rẽ kỳ vọng là

$$1 + \frac{1}{2} + \dots + \frac{1}{N} = O(\log N).$$

5.2.5 Thuật toán bốn

Ta tìm vết cận mọi hàng và cột có thể chứa điểm rẽ. Sau đó dùng quy hoạch động để giải truy vấn, trong đó chỉ những hàng và cột có khả năng chứa điểm rẽ mới được xem là trạng thái hợp lệ.

Theo phân tích kỳ vọng ở trên, số hàng và số cột cần xét đều là $O(\log N)$; kỳ vọng của tích cũng bằng tích của các kỳ vọng, nên số trạng thái kỳ vọng là $O(\log^2 N)$. Ngoài ra, ta lưu tổng tiền tố của hai dãy A và B để tăng tốc các phép chuyển trạng thái dọc theo một đoạn thẳng.

Độ phức tạp thời gian kỳ vọng là

$$O\left(T \cdot (N + \log^2 N)\right) = O(TN).$$

Cách này kỳ vọng đạt khoảng 50 điểm.

5.2.6 Thuật toán năm

Nút thắt của thuật toán trước là việc tìm các hàng và cột có khả năng chứa điểm rẽ. Bài toán con này có thể phát biểu như sau: trong một khoảng của một dãy, tìm các số nhỏ hơn mọi số ở phía trước hoặc nhỏ hơn mọi số ở phía sau. Chỉ cần biết, với mỗi vị trí, phần tử đầu tiên ở phía sau hoặc phía trước nhỏ hơn nó là đủ. Việc này có thể thực hiện bằng hàng đợi đơn điệu.

Chẳng hạn, để tìm phần tử đầu tiên ở phía sau nhỏ hơn mỗi phần tử, ta quét dãy từ phải sang trái. Với phần tử hiện tại, loại khỏi đuôi hàng đợi đơn điệu mọi phần tử không nhỏ hơn nó. Khi đó phần tử đầu tiên ở phía sau nhỏ hơn nó chính là phần tử ở đuôi hàng đợi hiện tại; sau đó đưa phần tử hiện tại vào hàng đợi.

Tiền xử lý chỉ cần thời gian tuyến tính. Mỗi truy vấn sau đó được trả lời trong thời gian kỳ vọng $O(\log^2 N)$.

Độ phức tạp thời gian kỳ vọng là

$$O(N + T \log^2 N).$$

Cách này kỳ vọng đạt 100 điểm. Tới đây bài toán đã được giải quyết trọn vẹn.

5.3 Lời cảm ơn

Xin cảm ơn CCF đã cung cấp nền tảng học tập và trao đổi.

Xin cảm ơn cô Kong Weiling và thầy Gu Fangming vì sự chỉ dẫn.

Xin cảm ơn các bạn học đã giúp đỡ tôi.

6 Thảo luận bài *Two strings*

Luo Gan

Trường Học Quân Hàng Châu, Chiết Giang

6.1 Mô tả bài toán

Cho hai chuỗi A và B . Có năm loại thao tác:

1. Thêm một ký tự vào đầu chuỗi A .
2. Thêm một ký tự vào cuối chuỗi A .
3. Thêm một ký tự vào đầu chuỗi B .
4. Thêm một ký tự vào cuối chuỗi B .
5. Hỏi số lần chuỗi B hiện tại xuất hiện trong chuỗi A hiện tại.

Bảo đảm mọi ký tự đều là chữ cái thường, và ban đầu hai chuỗi A, B đều không rỗng.

6.1.1 Dữ liệu vào

Dòng thứ nhất và dòng thứ hai lần lượt là chuỗi A và chuỗi B ban đầu. Dòng thứ ba chứa một số nguyên m , biểu diễn số thao tác. Tiếp theo là m dòng, mỗi dòng biểu diễn một thao tác. Số đầu tiên của mỗi dòng là một số trong khoảng 1–5; nếu giá trị này khác 5, sau số đó sẽ có một chữ cái thường.

6.1.2 Dữ liệu ra

Với mỗi truy vấn, in ra một số nguyên trên một dòng, biểu diễn số lần chuỗi B xuất hiện trong chuỗi A .

6.1.3 Quy mô dữ liệu

- Với 10% dữ liệu, tổng độ dài cuối cùng của hai chuỗi A và B không vượt quá 200, số thao tác không vượt quá 10.
- Với 30% dữ liệu, tổng độ dài cuối cùng của hai chuỗi A và B không vượt quá 2000, số thao tác không vượt quá 1000.

- Với 100% dữ liệu, tổng độ dài cuối cùng của hai chuỗi A và B không vượt quá 200000, số thao tác không vượt quá 200000.

6.2 Ví dụ

6.2.1 Dữ liệu vào

ababc

a

7

5

4 b

5

3 a

1 a

5

5

6.2.2 Dữ liệu ra

2

2

1

1

6.2.3 Giải thích ví dụ

Lần hỏi	Xâu A	Xâu B	Số lần B xuất hiện trong A
Lần 1	ababc	a	2
Lần 2	ababc	ab	2
Lần 3	aababc	aab	1
Lần 4	aababc	aab	1

6.3 Thuật toán đơn giản

6.3.1 Mô phỏng

Mô phỏng từng thao tác. Với mỗi truy vấn, trực tiếp liệt kê các vị trí mà chuỗi B có thể xuất hiện trong chuỗi A , rồi so sánh trực tiếp.

6.3.2 Mô phỏng tối ưu bằng KMP

Trên cơ sở mô phỏng, với mỗi truy vấn ta dùng thuật toán KMP để thực hiện phép khớp chuỗi.

6.4 Tối ưu mô phỏng bằng xử lý ngoại tuyến

Đọc trước toàn bộ thao tác liên quan tới chuỗi A , rồi ghép chúng thành một “chuỗi A lớn”.

Khi đó, ở mỗi thời điểm truy vấn, chuỗi A hiện tại tương đương với một đoạn nào đó của chuỗi A lớn. Bài toán trở thành hỏi trong đoạn đó có bao nhiêu chuỗi B .

Mỗi khi gặp thao tác loại 3 hoặc loại 4, ta chạy lại KMP một lần và ghi nhận những vị trí khớp với xâu B . Như vậy truy vấn được chuyển thành: trong các vị trí đó, có bao nhiêu vị trí nằm trong đoạn từ L tới $R - \text{Len}(B) + 1$.

Vì L giảm dần còn R tăng dần, mỗi truy vấn có thể được giải trong thời gian $O(1)$.

6.4.1 Độ phức tạp thời gian của từng thao tác

Thao tác	Độ phức tạp
Thao tác 1	$O(n) \rightarrow O(1)$
Thao tác 2	$O(n) \rightarrow O(1)$
Thao tác 3	$O(n)$
Thao tác 4	$O(n)$
Thao tác 5	$O(n) \rightarrow O(1)$

6.5 Thuật toán xử lý ngoại tuyến bằng mảng hậu tố

6.5.1 Từ trực tuyến sang ngoại tuyến

Sử dụng tư tưởng ngoại tuyến ở trên: lấy kết quả cuối cùng của hai xâu A, B , ghép chúng lại với nhau, đồng thời chèn một ký tự phân cách ở giữa, chẳng hạn $+$. Sau đó dùng mảng hậu tố để duy trì quan hệ giữa các hậu tố.

Ví dụ:

Xâu A lớn: ABABC
 Xâu B lớn: ABA
 Xâu cuối cùng: ABABC+ABA

Sau khi sắp xếp mảng hậu tố:

A
 ABA
 ABC+ABA
 ABABC+ABA
 BA
 BABC+ABA
 BC+ABA
 C+ABA
 +ABA

6.5.2 Tìm khoảng hậu tố khả thi

Xâu B hiện tại xuất hiện trong xâu A ở các vị trí từ i tới $i + \text{Len}(B) - 1$ khi và chỉ khi độ dài tiền tố chung dài nhất Lcp của xâu B hiện tại và hậu tố thứ i của xâu A thỏa mãn

$$Lcp \geq \text{Len}(B).$$

Ta có thể dùng phương pháp nhân đôi để tiền xử lý mảng height, rồi tìm khoảng hậu tố này trong thời gian $O(\log n)$.

6.5.3 Thống kê vị trí bắt đầu hợp lệ trong xâu A

Ta định nghĩa một vị trí hợp lệ như sau: với vị trí i trong xâu A lớn, toàn bộ đoạn

$$[i, i + \text{Len}(B) - 1]$$

đều nằm trong xâu A hiện tại.

6.5.4 Duy trì bằng cây BIT

Ta có thể nhận thấy: mỗi thao tác bất kỳ chỉ làm thay đổi tính hợp lệ của nhiều nhất hai vị trí. Vì vậy có thể dùng cây BIT để duy trì, trên một đoạn của mảng hậu tố, có bao nhiêu vị trí hợp lệ.

6.5.5 Độ phức tạp thời gian của từng thao tác

Thao tác	Độ phức tạp
Thao tác 1	$O(\log n)$
Thao tác 2	$O(\log n)$
Thao tác 3	$O(\log n)$
Thao tác 4	$O(\log n)$
Thao tác 5	$O(\log n)$

6.6 Các điểm kiểm tra

- Suy nghĩ sâu về bản chất của bài toán.
- Nắm vững nhiều loại cấu trúc dữ liệu.
- Khả năng tối ưu nhiều loại thuật toán mô phỏng.
- Khả năng vận dụng thuật toán ngoại tuyến.

7 *Catch The Penguins*

Zhang Wentao

Trường Học Quân Hàng Châu, Chiết Giang

7.1 Tóm tắt bài toán

Cho tọa độ của N con chim cánh cụt trong không gian bốn chiều.

Có Q truy vấn. Mỗi truy vấn cho một điểm và hỏi có bao nhiêu chim cánh cụt có tọa độ ở mọi chiều đều nhỏ hơn hoặc bằng tọa độ tương ứng của điểm đó.

Ví dụ vào.

```
1
0 0 0 0
2
1 1 1 0
1 1 1 -1
```

Ví dụ ra.

1
0

Quy mô dữ liệu. Có tất cả 20 bộ dữ liệu.

Dữ liệu	Ràng buộc
1–2	$N, Q \leq 5000$
3–6	$N, Q \leq 10000$
7–14	$N \leq 30000, Q \leq 10000$
15–18	$N \leq 10000, Q \leq 30000$
19–20	$N, Q \leq 30000$

7.2 Thuật toán 0

Trực tiếp vét cạn. Với mỗi truy vấn, dùng một vòng lặp for để liệt kê từng chim cánh cụt, rồi kiểm tra xem cả bốn tọa độ của nó có đều nhỏ hơn hoặc bằng tọa độ của truy vấn hay không.

Độ phức tạp thời gian là $O(n^2)$.

Kỳ vọng đạt 10 điểm.

7.3 Tối ưu thuật toán 0

Sắp xếp riêng các chim cánh cụt theo từng chiều trong bốn chiều tọa độ.

Với mỗi truy vấn, trên mỗi chiều ta tìm nhị phân trong mảng đã sắp xếp tương ứng để biết số chim cánh cụt có tọa độ ở chiều đó nhỏ hơn hoặc bằng tọa độ của truy vấn. Lấy giá trị nhỏ nhất trong bốn kết quả này để chọn chiều đem ra liệt kê, như một tối ưu hằng số.

Kỳ vọng đạt 30 điểm.

7.4 Tiếp tục tối ưu?

Nếu muốn dùng thuật toán trực tuyến, dường như không có cấu trúc dữ liệu phù hợp để trả lời truy vấn tọa độ bốn chiều, nên rất khó tiếp tục theo hướng đó.

Ta hãy thử chuyển sang xét cách xử lý truy vấn ngoại tuyến.

7.5 Thuật toán chia khối

Khi đã có ý tưởng ngoại tuyến, kết hợp thêm tư tưởng chia khối thì không khó nghĩ ra một thuật toán có độ phức tạp tốt hơn một chút so với vét cạn.

7.5.1 Rời rạc hóa và xử lý ngoại tuyến

Sắp xếp riêng các truy vấn và các chim cánh cụt theo tọa độ chiều thứ nhất.

Sau đó liệt kê các truy vấn.

7.5.2 Với truy vấn đang được liệt kê

Thêm vào tất cả các chim cánh cụt có tọa độ chiều thứ nhất nhỏ hơn hoặc bằng tọa độ chiều thứ nhất của truy vấn.

Sắp xếp theo chiều thứ hai. Mỗi khi thêm $\sqrt{n}$ chim cánh cụt thì thực hiện một lần sắp xếp bằng vết cạn; phần còn lại cũng xử lý vết cạn.

Độ phức tạp của phần này là $O(n\sqrt{n})$.

7.5.3 Khi thực hiện một lần sắp xếp vết cạn

Giả sử số chim cánh cụt đã được thêm vào và đã sắp xếp là M .

Ta chia khối, sắp xếp theo tọa độ chiều thứ ba, rồi theo chiều thứ tư duy trì cây đoạn lưu trọng số tiền tố.

Cụ thể, có thể dùng cây đoạn hàm thức.

7.5.4 Truy vấn

Với các khối đầy đủ: dùng nhị phân cộng truy vấn.

Với các phần không trọn khối: xử lý vết cạn.

Tổng độ phức tạp là $O(n\sqrt{n} \log n)$.

Kỳ vọng đạt 60–95 điểm.

7.6 Thuật toán tốt hơn

Thông thường, xử lý chia khối là một phương án thay thế hiệu quả khi tồn tại thuật toán có độ phức tạp tốt hơn nhưng độ phức tạp tư duy lại khá cao. Ở đây, chỉ cần suy nghĩ thêm một chút là ta có thể tìm được một thuật toán chia để trị tốt hơn.

7.6.1 Bước 1

Sắp xếp chung các truy vấn và chim cánh cụt theo tọa độ chiều thứ nhất.

Sau đó thực hiện “trộn” theo tọa độ chiều thứ hai.

7.6.2 Bước 2

Giả sử dãy chỉ số sau khi sắp xếp là $id[L]$ tới $id[R]$, trong đó có phần tử là chim cánh cụt và có phần tử là truy vấn. Gọi MID là điểm cuối của đoạn bên trái.

7.6.3 Bước 3

Đệ quy xử lý đáp án của hai phần $L-MID$ và $MID+1-R$. Dĩ nhiên, khi L và R bằng nhau thì có thể trả về ngay.

7.6.4 Bước 4

Ta thấy trường hợp chưa được xử lý chỉ còn là ảnh hưởng của chim cánh cụt ở nửa trái, tức $L-MID$, lên truy vấn ở nửa phải, tức $MID+1-R$. Lúc này cả hai nửa đều đã được sắp xếp theo tọa độ chiều thứ hai (lý

do được giải thích ngay sau đây), và thứ tự theo chiều thứ nhất của chúng không còn cần thiết nữa.

Ta trộn nửa trái và nửa phải theo tọa độ chiều thứ hai, tức gộp hai bảng tuyến tính đã có thứ tự. Khi tạo bảng mới, nếu phần tử được thêm là chim cánh cụt ở nửa trái, ta chèn điểm tạo bởi tọa độ chiều thứ ba và chiều thứ tư của nó vào một “tập hợp”. Nếu phần tử được thêm là truy vấn ở nửa phải, ta hỏi trong “tập hợp” có bao nhiêu điểm nằm ở góc dưới bên trái của điểm tạo bởi hai tọa độ chiều thứ ba và chiều thứ tư của truy vấn đó, rồi cộng số này vào đáp án của truy vấn.

7.6.5 Bước 5

Về cách cài đặt “tập hợp”, ta có thể dùng cây BIT (hoặc cây đoạn) lồng cây cân bằng.

Tổng độ phức tạp thời gian là $O(n \log^3 n)$.

7.7 Các điểm kiểm tra

- Chuyển đổi tư duy từ trực tuyến sang ngoại tuyến.
- Vận dụng các tư tưởng chia khối, chia để trị.
- Xử lý thứ tự dữ liệu một cách khéo léo.
- Nắm vững cấu trúc dữ liệu.